

MYSQL mock-2

1. Find the Total Revenue Generated from Ticket Sales

The screenshot shows a MySQL database interface with a query editor on the left and a results pane on the right. The query editor contains the following SQL code:

```
1 SELECT *
2 FROM Booking
3 ORDER BY booking_datetime DESC
4 LIMIT 5 OFFSET 0;
5 SELECT * FROM Booking;
6
7 SELECT sum(total_cost) AS revenue_generated FROM booking;
8
```

The results pane shows the output of the query, which is a single row with the value 829417.12.

revenue_generated
829417.12

2. Get All Reviews for a Specific Movie

The screenshot shows a MySQL database interface with a query editor on the left and a results pane on the right. The query editor contains the following SQL code:

```
1 SELECT sum(total_cost) AS total_revenue
2 FROM Booking;
3
4 SELECT reviewer_name, review_id, content FROM Review WHERE movie_id=403;
5 SELECT user_id, name FROM User WHERE email IS NULL;
6
```

The results pane shows the output of the query, which is a table of reviews for movie_id 403.

reviewer_name	review_id	content
Andrea Ortega	24064	Average movie. Thought any look serve away bet...
David Mendoza	24065	Absolutely amazing! Seem black service sure bo...
Scott Glover	24066	Terrible movie! Hundred report born control exist...
Carolyn Zhang	24067	Would recommend to friends. Sport time build one...
Susan Blair	24068	Would recommend to friends. Record eat ask hap...
James Gomez	24069	Waste of time and money. Go exactly bed phone ...
Benjamin Odom	24070	Could have been better. Receive goal save social...
Dorothy Henderson	24071	Could have been better. Truth training create mil...
Lauree Hines	24072	Really enjoyed it Police billion it pick answer pag...
Karen Armstrong	24073	Masterpeace! World ground business how thank a...
Janet Jordan	24074	Could have been better. Ping! 189y wide single...

3. Find Customers Who Haven't Provided an Email

The screenshot shows a MySQL database interface with a query editor on the left and a results pane on the right. The query editor contains the following SQL code:

```
1 SELECT sum(total_cost) AS total_revenue
2 FROM Booking;
3
4 SELECT reviewer_name, review_id, content FROM Review WHERE movie_id=403;
5 SELECT user_id, name FROM User WHERE email IS NULL;
6
```

The results pane shows the output of the query, which is a table of users who have not provided an email.

user_id	name
---------	------

4. Find the Total Revenue Generated from Ticket Sales

The screenshot shows a database IDE with a SQL query editor and a results pane. The query is as follows:

```
1 SELECT *
2 FROM Booking
3 ORDER BY booking_datetime DESC
4 LIMIT 5 OFFSET 0;
5 SELECT * FROM Booking;
6
7 SELECT sum(total_cost) AS revenue_generated FROM booking;
8
```

The results pane shows a single row with the value 8229417.12 for the revenue_generated column.

5. Retrieve the Average Movie Rating

The screenshot shows a database IDE with a SQL query editor and a results pane. The query is as follows:

```
1 SELECT * FROM Movie;
2 SELECT avg(rating) AS average_movie_rating FROM Movie;
```

The results pane shows a single row with the value 5.5678000000000005 for the average_movie_rating column.

6. Analyze Booking Trends by Day of the Week

The screenshot shows a database IDE with a SQL query editor and a results pane. The query is as follows:

```
1 SELECT
2 CASE strftime('%w', booking_datetime)
3 WHEN '0' THEN 'Sunday'
4 WHEN '1' THEN 'Monday'
5 WHEN '2' THEN 'Tuesday'
6 WHEN '3' THEN 'Wednesday'
7 WHEN '4' THEN 'Thursday'
8 WHEN '5' THEN 'Friday'
9 WHEN '6' THEN 'Saturday'
10 END AS day_of_week,
11 COUNT(booking_id) AS total_bookings
12 FROM Booking
13 GROUP BY strftime('%w', booking_datetime)
14 ORDER BY strftime('%w', booking_datetime);
```

The results pane shows a table with two columns: day_of_week and total_bookings. The data is as follows:

day_of_week	total_bookings
Sunday	2042
Monday	2213
Tuesday	2513
Wednesday	2269
Thursday	1943
Friday	1946
Saturday	2021